

Document number: P1943R0
Date: 2019-10-07
Project: Programming Language C++
Audience: LEWG, SG1
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Networking TS changes to improve completion token flexibility and performance

Introduction

The Networking TS, and the Asio library on which it is based, uses *completion tokens* as a mechanism to customise the asynchronous API surface of the library. Completion tokens, which were first released in 2013 as part of Boost 1.54, automatically adapt asynchronous operations to the look and feel of the user's preferred coordination mechanism. For example, the `use_future` token causes operations to produce a `std::future`; Asio's `use_awaitable` token makes operations produce awaitables that are compatible with the Coroutines TS; and Asio's `yield_context` or Boost.Fiber's `yield` provide a synchronous experience on top of coroutines and fibers. The response from users has been overwhelmingly positive.

However, completion tokens as currently specified have some limitations that constrain their support for, and efficiency in, certain use cases. In particular:

- The underlying asynchronous operations are always launched immediately, and there is no support for deferring the launch of the operation. Consequently, some form of synchronisation is required to prevent a race between suspension and resumption, when using coroutines or fibers.
- Many asynchronous operations allocate memory to store some intermediate state, and allocators may be used to optimise this cost away. However, there is no way to determine the memory requirements at compile time, and then to use this information to automatically pre-allocate suitable storage.

This paper proposes the following changes to the completion token mechanism:

- Give concrete completion token implementations control over when and where an operation is initiated.
- Permit completion token implementations to incorporate operation initiation into the function return type, without type erasure.
- Introduce concepts to improve completion token usability.
- Allow completion token implementations to determine an operation's intermediate memory storage requirements.

Overview of proposed changes

New form for the `async_result` customisation point

These changes alter `async_result` to a simpler form with only a static member function named `initiate()`:

```
template <class CompletionToken, class Signature>
struct async_result
{
    template<
        class Initiation,
        class RawCompletionToken,
        class... Args
    >
    static initiating-fn-return-type initiate(
        Initiation&& initiation,
        RawCompletionToken&& token,
        Args&&... args
    );
};
```

An `async_result` specialisation's implementation of the `initiate()` static member function must:

- Transform the completion token into a completion handler object handler.
- Cause the invocation of the function object initiation as if by calling:

```
std::forward<Initiation>(initiation)(
    std::move(handler),
    std::forward<Args>(args)...);
```

The invocation of `initiation` may be immediate, or it may be deferred (e.g. to support lazily evaluation). If initiation is deferred, the `initiation` and `args` objects must be decay-copied and moved as required.

A helper function template `async_initiate` has also been added as a wrapper for the invocation of `async_result<>::initiate`:

```
template<
    class CompletionToken,
    completion_signature Signature,
    class Initiation,
    class... Args
>
DEDUCED async_initiate(
    Initiation&& initiation,
    CompletionToken& token,
    Args&&... args
);
```

This change to `async_result` gives concrete completion token implementations control over when and where an operation is initiated. This enables use cases such as:

- Coroutines and fibers can defer initiation until after suspension. This eliminates a potential race between suspension and resumption that must otherwise be addressed using synchronisation.
- Users can specify a completion token that guarantees that operation initiation always occurs within the context of a particular executor, such as a strand.
- Lazy initiation of operations, by capturing the initiation object and arguments into the return type.

In addition, this change also permits completion token implementations to incorporate the initiation object (and its arguments) into the function return type, without type erasure.

Introduce new concepts

This change introduces three new concepts:

- `template <class T> concept completion_signature = /*...*/;`
Checks if `T` is a signature of the form `R(Args...)`.
- `template <class T, completion_signature Signature> concept completion_handler_for = /*...*/;`
Checks if `T` is usable as a completion handler with the specified signature.
- `template <class T, completion_signature Signature> concept completion_token_for = /*...*/;`
Checks if `T` is a completion token that can be used with `async_initiate` and the specified signature.

These concepts improve user experience by checking constraints at the API boundary. They also make asynchronous operation declarations self-documenting with respect to the operation's completion signature, as in:

```
template<
    class AsyncReadStream,
    class MutableBufferSequence,
    completion_token_for<void(error_code, size_t)>&& CompletionToken
>
DEDUCED async_read(
    AsyncReadStream& s,
    const MutableBufferSequence& buffers,
    CompletionToken&& token
);
```

Add a trait to determine intermediate storage requirements

As specified in the Networking TS section **[`async.reqmts.async.alloc`]**, asynchronous operations may allocate memory and, if they do, will use the completion handler's associated allocator. To support the pre-allocation of correctly sized memory, these changes add a new trait `intermediate_storage`.

```
template<class T, class... Args>
struct intermediate_storage
{
    typedef see-below type;
};

template<class T, class... Args>
using inter_storage_t = typename intermediate_storage<T, Args...>::type;
```

The primary template is defined such that:

- If `T` has a nested template type `type<Args...>`, type is `T::type<Args...>`.
- Otherwise, type is `void`.

The trait may be specialised for user-defined types of `T`.

When type is non-void, it shall be a trivial standard-layout type suitable for use as uninitialised storage by the operation initiated by the type T. The type shall be non-void only if an operation allocates fixed size memory, with at most one allocated object extant at any one time. If this requirement is not met, type shall be void. The trait may then be applied to the Initiation object passed to a concrete completion token's `async_result` specialisation, and this requirement enables a trivial allocator implementation that simply returns the address of the pre-allocated storage.

Asynchronous operations are not required to specialise this trait. However, implementation experience in Asio indicates that all operations defined within the TS can satisfy the requirement of this trait, and we may wish to mandate that they do.

Importantly, this change places no burden on a user to maintain pre-allocated storage, unless they wish to do so. Users can continue to simply launch asynchronous operations without being exposed to the asynchronous lifetime requirements of the operation's underlying state.

Impact of proposed changes

Impact on users of asynchronous operations

Existing code, where asynchronous operations are consumed using the currently available completion tokens, is unaffected.

For new code, users of asynchronous operations will have a wider set of completion token types and capabilities available to them. Some examples of these new completion tokens are illustrated in the Appendix below.

Impact on authors of asynchronous operations

In order to illustrate the impact on authors of asynchronous operations, the following table shows the changes to the example from Networking TS section `[async.reqmts.async.return.value]`.

Before	After
Given an asynchronous operation with Completion signature <code>void(R1 r1, R2 r2)</code>, an initiating function meeting these requirements may be implemented as follows: <pre>template<class CompletionToken> auto async_xyz(T1 t1, T2 t2, CompletionToken&& token) { typename async_result<decay_t<CompletionToken>, void(R1, R2)>::completion_handler_type completion_handler(forward<CompletionToken>(token)); async_result<decay_t<CompletionToken>, void(R1, R2)> result(completion_handler); // initiate the operation and cause completion_handler // to be invoked with the result return result.get(); }</pre>	Given an asynchronous operation with Completion signature <code>void(R1 r1, R2 r2)</code>, an initiating function meeting these requirements may be implemented as follows: <pre>template<completion_token_for<void(R1, R2)> CompletionToken> auto async_xyz(T1 t1, T2 t2, CompletionToken&& token) { return async_result<decay_t<CompletionToken>, void(R1, R2)>::initiate([](auto completion_handler, T1 t1, T2 t2) { // initiate the operation and cause completion_handler // to be invoked with the result }, forward<CompletionToken>(token), t1, t2); }</pre>
For convenience, initiating functions may be implemented using the <code>async_completion</code> template: <pre>template<class CompletionToken> auto async_xyz(T1 t1, T2 t2, CompletionToken&& token) { async_completion<CompletionToken, void(R1, R2)> init(token); // initiate the operation and cause init.completion_handler // to be invoked with the result return init.result.get(); }</pre>	For convenience, initiating functions may be implemented using the <code>async_initiate</code> function template: <pre>template<completion_token_for<void(R1, R2)> CompletionToken> auto async_xyz(T1 t1, T2 t2, CompletionToken&& token) { return async_initiate<CompletionToken, void(R1, R2)>([](auto completion_handler, T1 t1, T2 t2) { // initiate the operation and cause completion_handler // to be invoked with the result }, token, t1, t2); }</pre>

Impact on authors of completion tokens

As an example, let us consider a simple completion token:

```
constexpr struct log_result_t {} log_result;
```

that is used as follows:

```
my_timer.async_wait(log_result);
```

and which logs the result of an operation to `std::cout`. The following table illustrates the implementation of the completion token before and after the proposed changes.

Before	After
<pre>template <class R, class... Args> class async_result<log_result_t, R(Args...)> { public: struct completion_handler_type { completion_handler_type(log_result_t) { } void operator()(Args... args) { std::cout << "Result:"; ((std::cout << " " << args), ...); std::cout << "\n"; } }; using return_type = void; explicit async_result(completion_handler_type&) { } return_type get() { } };</pre>	<pre>template <class R, class... Args> struct async_result<log_result_t, R(Args...)> { template <class Initiation, class... InitArgs> static void initiate(Initiation initiation, log_result_t, InitArgs&&... init_args) { initiation([] (Args... args) { std::cout << "Result:"; ((std::cout << " " << args), ...); std::cout << "\n"; }, std::forward<InitArgs>(init_args)...); } };</pre>

Implementation experience

The first of the proposed changes:

- Give concrete completion token implementations control over when and where an operation is initiated.

has been implemented in Asio 1.14.0, which was shipped as part of the Boost 1.70 release. The remaining three changes are currently implemented on branches of Asio and Boost.Asio.

Appendix: Example completion tokens

A token to make any operation lazy

The lazy completion token transforms any asynchronous operation into a function object. This function object can be stored and used to launch the operation at a later time, using another completion token.

Usage example

```
auto lazy_op = my_timer.async_wait(lazy);
// ...
lazy_op([](auto...){ /*...*/ });
```

Completion token definition

```
constexpr struct lazy_t {} lazy;

template <completion_signature Signature>
struct async_result<lazy_t, Signature>
{
    template <typename Initiation, typename... InitArgs>
    static auto initiate(Initiation initiation,
                        lazy_t, InitArgs... init_args)
    {
        return [
            initiation = std::move(initiation),
            init_arg_pack = std::make_tuple(std::move(init_args)...)
        ](auto&& token) mutable
        {
            return std::apply(
                [&](auto&&... args)
                {
                    return async_initiate<decltype(token), Signature>(
                        std::move(initiation), token,
                        std::forward<decltype(args)>(args)...);
                },
                std::move(init_arg_pack)
            );
        }
    };
};
```

```

    };
}
};

```

Support for fibers

The `use_fiber` completion token provides native support for fibers via the `boost::context::fiber` class (on which the proposed standards fibers are based).

Usage example

```

boost::context::fiber echo(tcp::socket socket, boost::context::fiber f)
{
    for (;;)
    {
        char data[1024];
        auto [e1, n] = socket.async_read_some(buffer(data), use_fiber(f));
        if (e1) break;
        auto [e2, _] = async_write(socket, buffer(data, n), use_fiber(f));
        if (e2) break;
    }
    return f;
}

```

Completion token definition

```

struct use_fiber
{
    explicit use_fiber(boost::context::fiber& f) : fiber(f) {}
    boost::context::fiber& fiber;
};

template <typename R, typename... Args>
class async_result<use_fiber, R(Args...)>
{
public:
    template <typename Initiation, typename... InitArgs>
    static auto initiate(Initiation&& initiation,
        use_fiber u, InitArgs&&... init_args)
    {
        std::tuple<Args...>* result_ptr;

        u.fiber = std::move(u.fiber).resume_with(
            [&](boost::context::fiber f)
            {
                std::forward<Initiation>(initiation)(
                    [&, f = std::move(f)](Args... results) mutable
                    {
                        std::tuple<Args...> result(std::move(results)...);
                        result_ptr = &result;
                        std::move(f).resume();
                    },
                    std::forward<InitArgs>(init_args)...
                );

                return boost::context::fiber{};
            }
        );

        return std::move(*result_ptr);
    }
};

```

Support for coroutines

The `use_await` token allows operations to be used with arbitrary coroutines. This example illustrates how the operation's initiation object is captured within the awaitable without type erasure, as well using the `intermediate_storage` trait to reserve space within the awaitable object for use by the operation.

Usage example

Note: `simple_coro` is a user-defined coroutine wrapper with a corresponding user-defined promise implementation.

```

simple_coro listener(tcp::acceptor& acceptor)
{
    for (;;)
    {
        if (auto [_, socket] = co_await acceptor.async_accept(use_await); socket.is_open())
        {
            for (;;)
            {

```

```

        char data[1024];
        auto [e1, n] = co_await socket.async_read_some(buffer(data), use_await);
        if (e1) break;
        auto [e2, _] = co_await async_write(socket, buffer(data, n), use_await);
        if (e2) break;
    }
}
}
}

```

Completion token definition

```

constexpr struct use_await_t {} use_await;

template <typename R, typename... Args>
class async_result<use_await_t, R(Args...)>
{
private:
    template <typename Initiation, typename... InitArgs>
    struct awaitable
    {
        template <typename T>
        class allocator;

        struct handler
        {
            typedef allocator<void> allocator_type;

            allocator_type get_allocator() const
            {
                return allocator_type(nullptr);
            }

            void operator()(Args... results)
            {
                std::tuple<Args...> result(std::move(results)...);
                awaitable_>result_ = &result;
                coro_.resume();
            }

            awaitable* awaitable_;
            std::experimental::coroutine_handle<> coro_;
        };

        using storage_type = intermediate_storage_t<Initiation, handler, InitArgs...>;

        template <typename T>
        class allocator
        {
        public:
            typedef T value_type;

            explicit allocator(awaitable* a) noexcept
                : awaitable_(a)
            {
            }

            template <typename U>
            allocator(const allocator<U>& a) noexcept
                : awaitable_(a.awaitable_)
            {
            }

            T* allocate(std::size_t n)
            {
                if constexpr (std::is_same_v<storage_type, void>)
                {
                    return static_cast<T*>(::operator new(sizeof(T) * n));
                }
                else
                {
                    return static_cast<T*>(static_cast<void*>(&awaitable_>storage_));
                }
            }

            void deallocate(T* p, std::size_t)
            {
                if constexpr (std::is_same_v<storage_type, void>)
                {
                    ::operator delete(p);
                }
            }

        private:
            template <typename> friend class allocator;
            awaitable* awaitable_;
        };
    };
};

```

```

};

bool await_ready() const noexcept
{
    return false;
}

void await_suspend(std::experimental::coroutine_handle< h> h) noexcept
{
    std::apply(
        [&](auto&&... a)
        {
            initiation_(handler{this, h}, std::forward<decltype(a)>(a)...);
        },
        init_args_
    );
}

std::tuple<Args...> await_resume()
{
    return std::move(*static_cast<std::tuple<Args...*>(result_));
}

Initiation initiation_;
std::tuple<InitArgs...> init_args_;
void* result_ = nullptr;
std::conditional_t<std::is_same_v<storage_type, void>, char, storage_type> storage_{};
};

public:
    template <typename Initiation, typename... InitArgs>
    static auto initiate(Initiation initiation,
        use_await_t, InitArgs... init_args)
    {
        return awaitable<Initiation, InitArgs...>{
            std::move(initiation),
            std::forward_as_tuple(std::move(init_args)...)};
    }
};

```